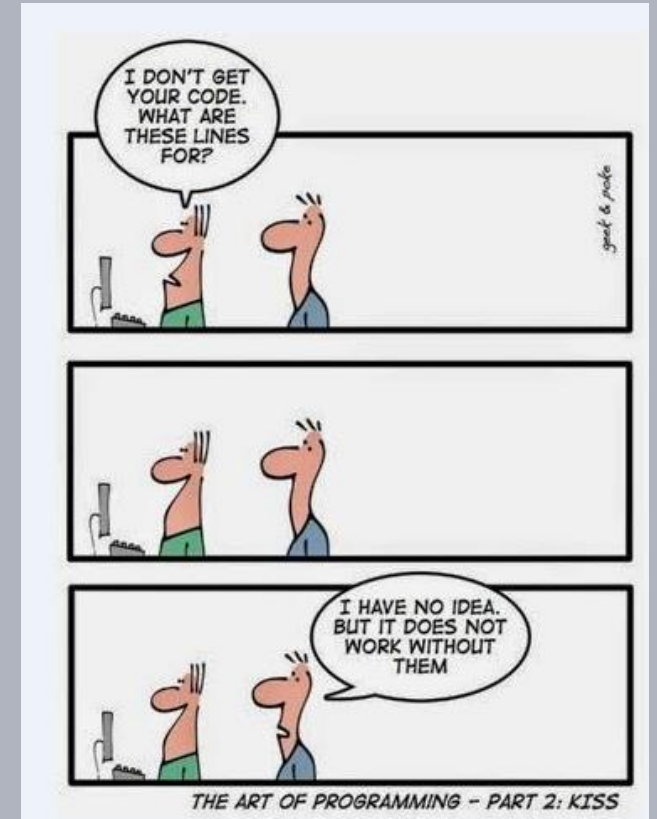


Advanced Software Design Techniques

Introduction

Prof. Dr.-Ing. Peter Fromm



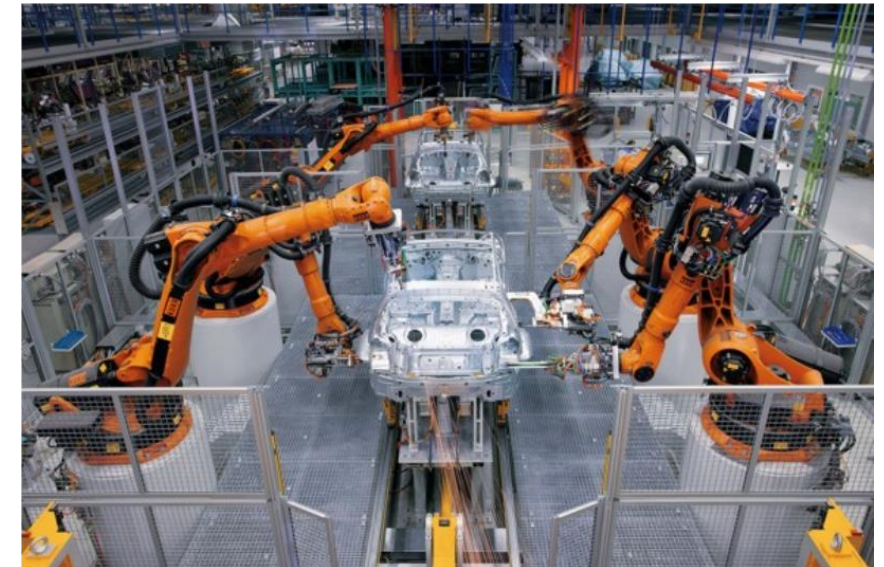
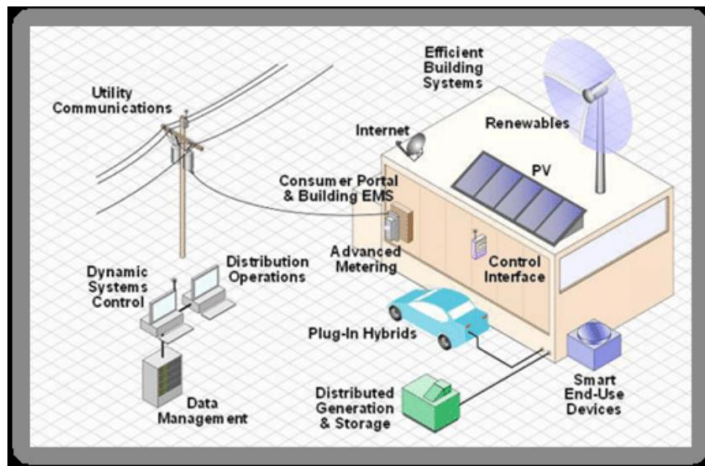
Content

- Programming – How it all started
- A short history of programming Languages
- The software crisis
- Software Engineering
- Why object-orientation?
- Modelling and UML

Innovation through Software



80% of today's
industrial innovation
is generated by
software



Software Development

Level 3: Design

Mastering the design
„We can improve this....“

What makes the
difference...

Level 2:
Functionality

Mastering the requirements
„Well, something happens....“

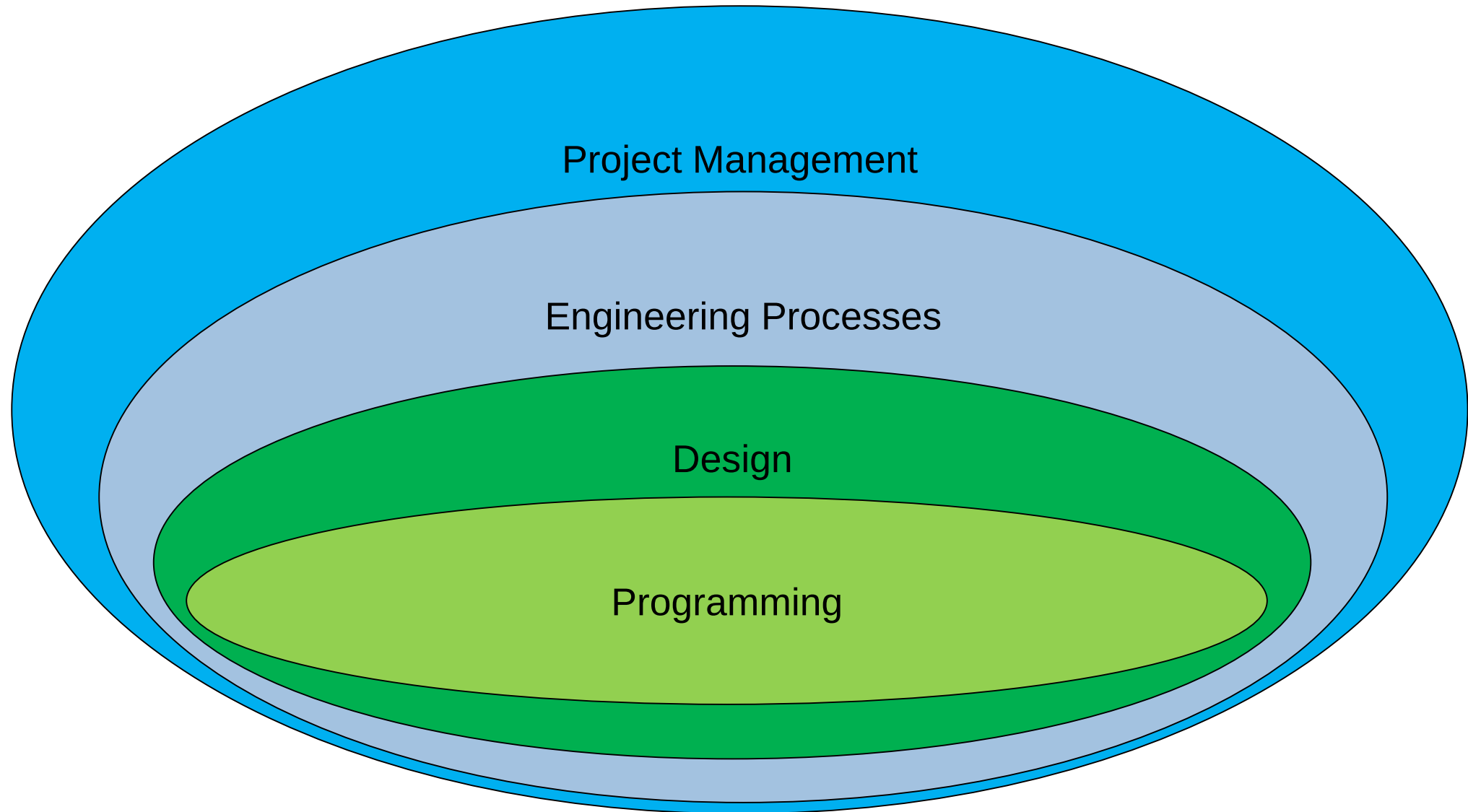
How to find them...

Level 1: Syntax

Mastering the syntax, compiler, debugger
„What the hack....“

Basic stuff...

All about Programming?



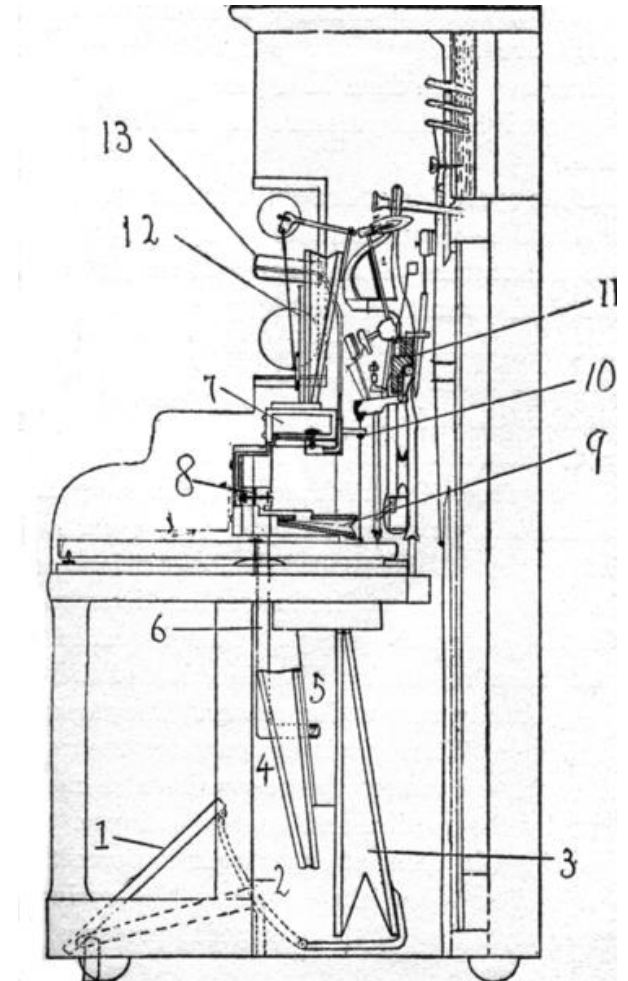
Where it all started...

- Jacquard loom (punched cards)



Where it all started...

- Church bells
 - Pinned barrels in use since several centuries
- The Player Piano
 - Presented 1876 at the Philadelphia Centennial Exposition
 - Provided controlled note duration and percussive striking
 - “Program” stored on paper rolls (12 and 13)



What is a program?

- Inventors of shown devices probably wouldn't have called the configuration a program, but ...
- *„A **program** is a list of instructions written in a programming language that is used to control the behavior of a machine, often a computer (in this case it is known as a computer program). (Note that 'program' (not 'programme') is the standard spelling for this sense of the word in all versions of English.)“* (Source: Wikipedia)
- *„**Computation** is a general term for any type of process, algorithm or measurement; this often includes but is not limited to digital data. This includes phenomena ranging from human thinking to calculations with a more narrow meaning. Computation is a process following a well-defined model that is understood and can be expressed in an algorithm, protocol, network topology, etc. Computation is also a major subject matter of computer science: it investigates what can or cannot be done in a computational manner.“* (Source: Wikipedia)

The first programmer

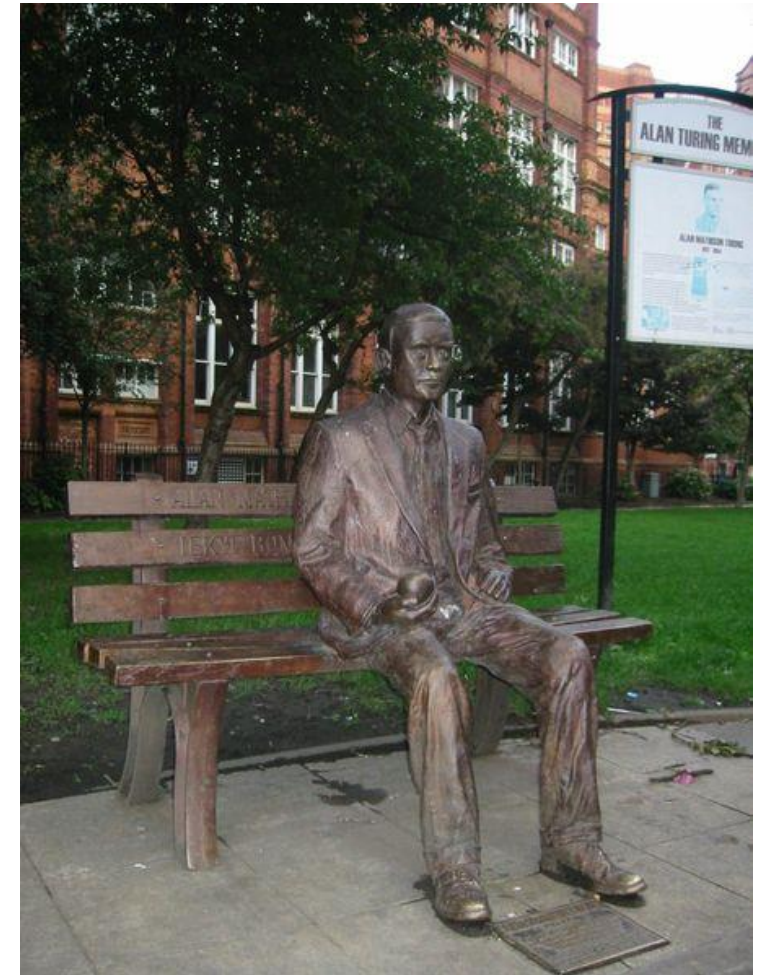
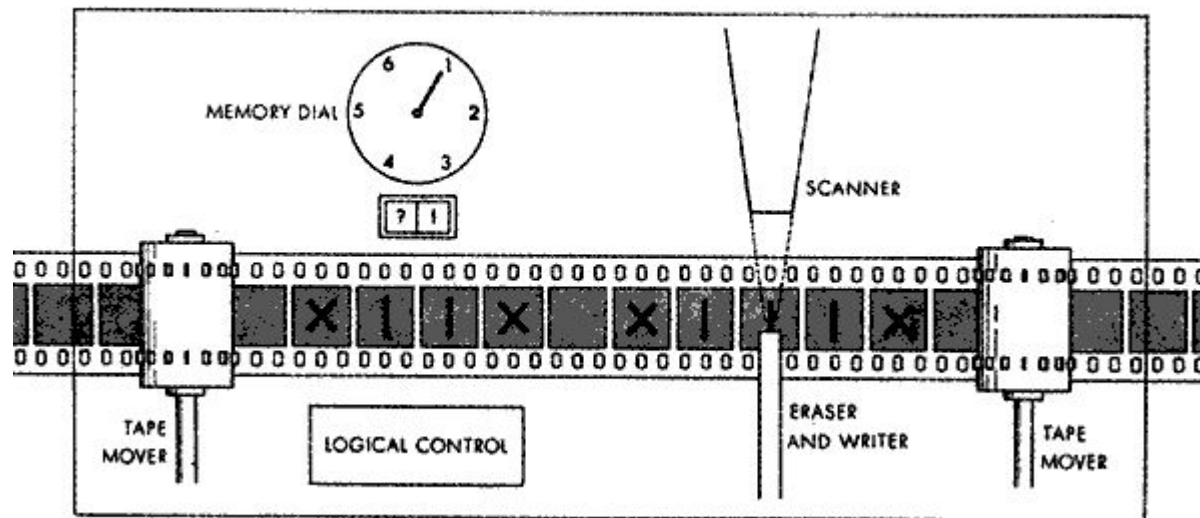
- Ada Lovelace (1815 – 1852)
- 1842 Translation of Luigi Menabrea's algorithm of Bernoulli numbers on Charles Babbage's "Analytical Engine" → the first ever written computer program
- Unfortunately, the Analytical Engine never was built



The first Computer

Turing Machine

- Developed by Alan Turing (1912 – 1954), an English mathematician
- Not a real machine, but a thought experiment about mechanical computing
- First presented in 1936
- Despite its theoretical approach, a Turing machine yields many insights into computer science and theory

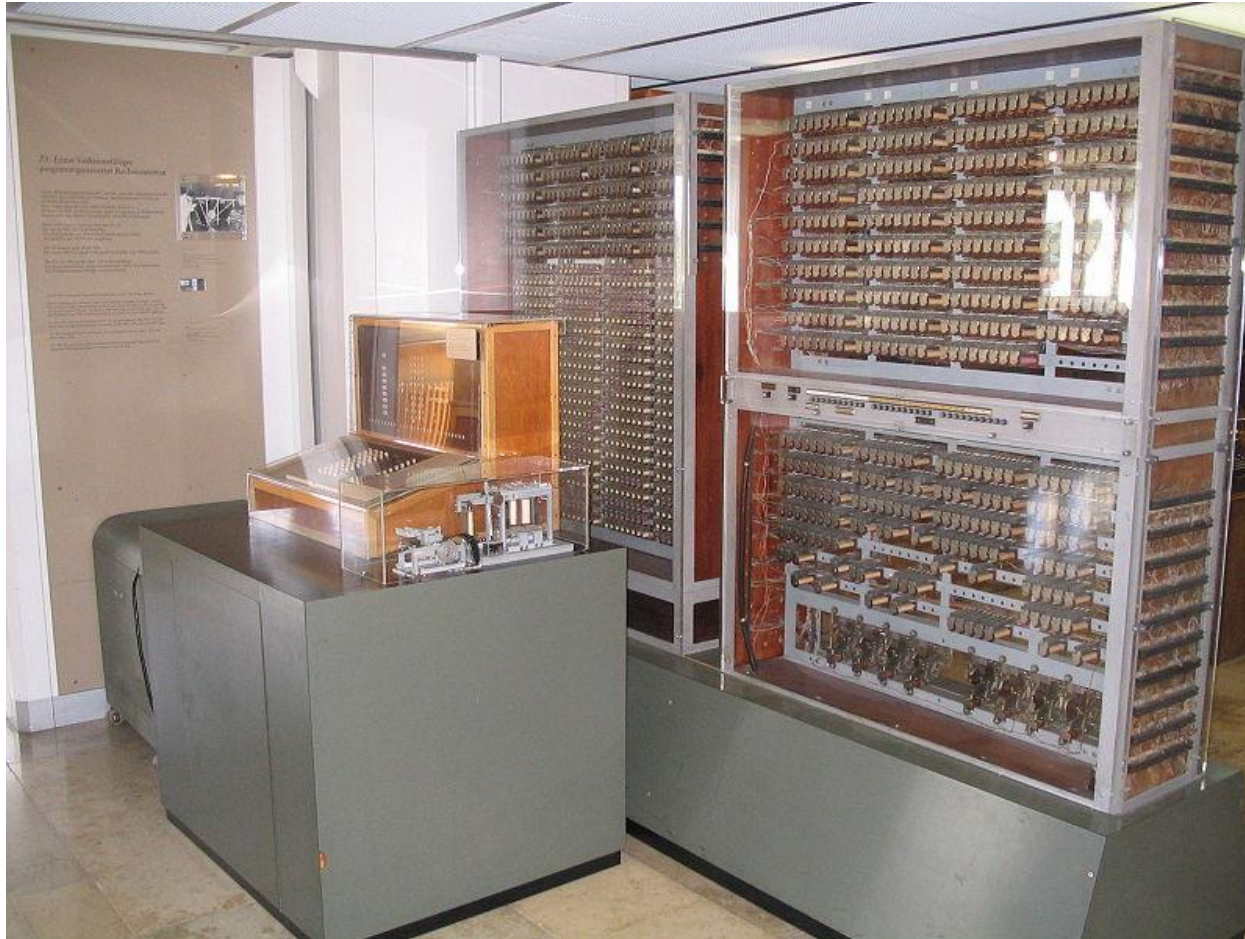


1940 – 1950

The first experimental hardware

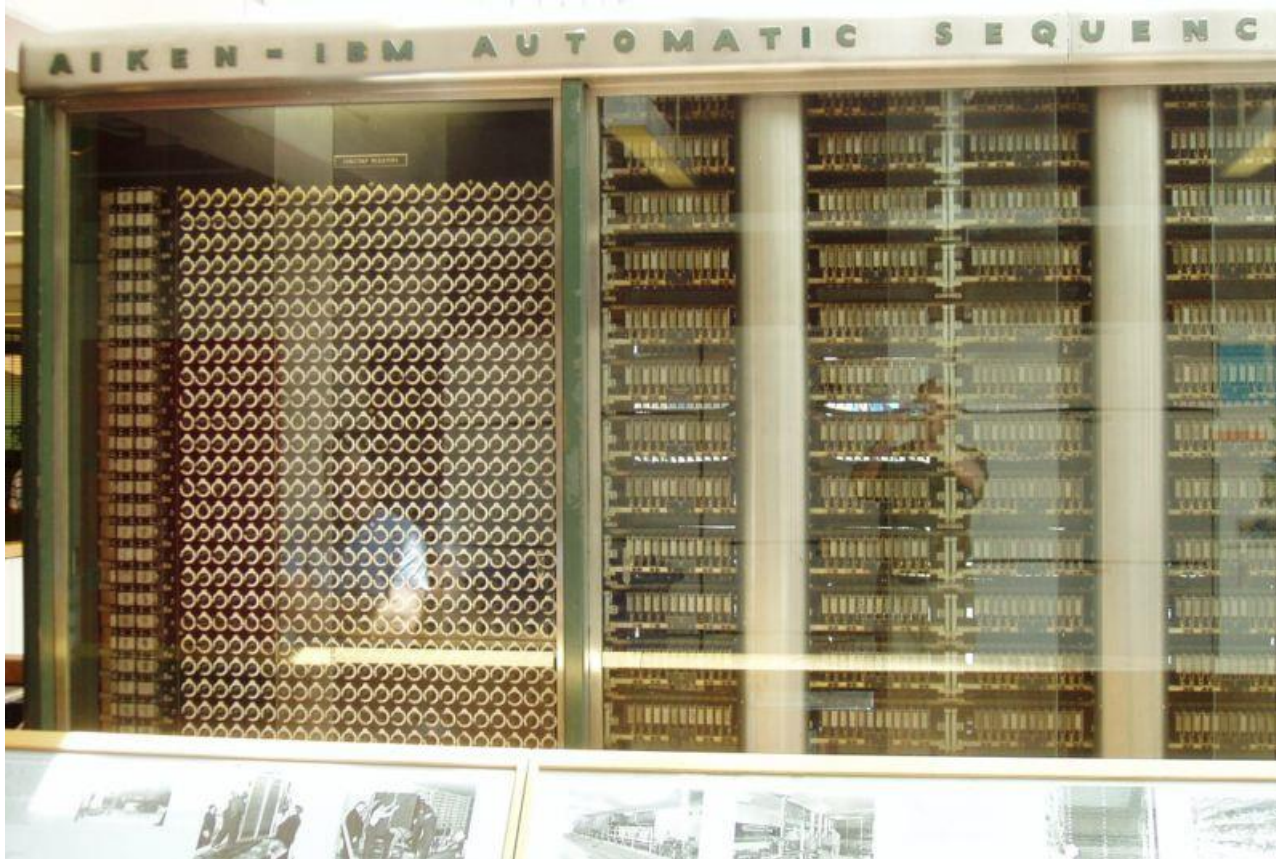
Name	First operation	Technology	Programming
Zuse Z3 (Germany)	1941	Electro-Mechanical	Punched film stock
Harward Mark 1 – IBM (US)	1944	Electro-Mechanical	24 channel punched paper type
ENIAC (US)	1945	Electronic	Patch cables and switches, later ROM
EDSAC (UK)	1949	Electronic	Delay line memory
Manchester Mark 1 (UK)	1949	Electronic	Cathode ray memory, magnetic drum

Zuse Z3



Konrad Zuse
(1910 – 1995)

IBM Mark 1



Mark 1 – the first bug

9/9

0800 Antan started

1000 " stopped - antan ✓

1300 (033) MP-MC ~~1.58264000~~ 2.130476415 ~~4.615925059(-2)~~ { 1.2700 9.037847025 9.037846995 correct

(033) PRO 2 2.130476415

correct 2.130676415

Relays 6-2 in 033 failed spiral speed test in relay

Relays changed

1100 Started Cosine Tape (Sine check)

1525 Started Multy Adder Test.

1545  Relay #70 Panel F (moth) in relay.

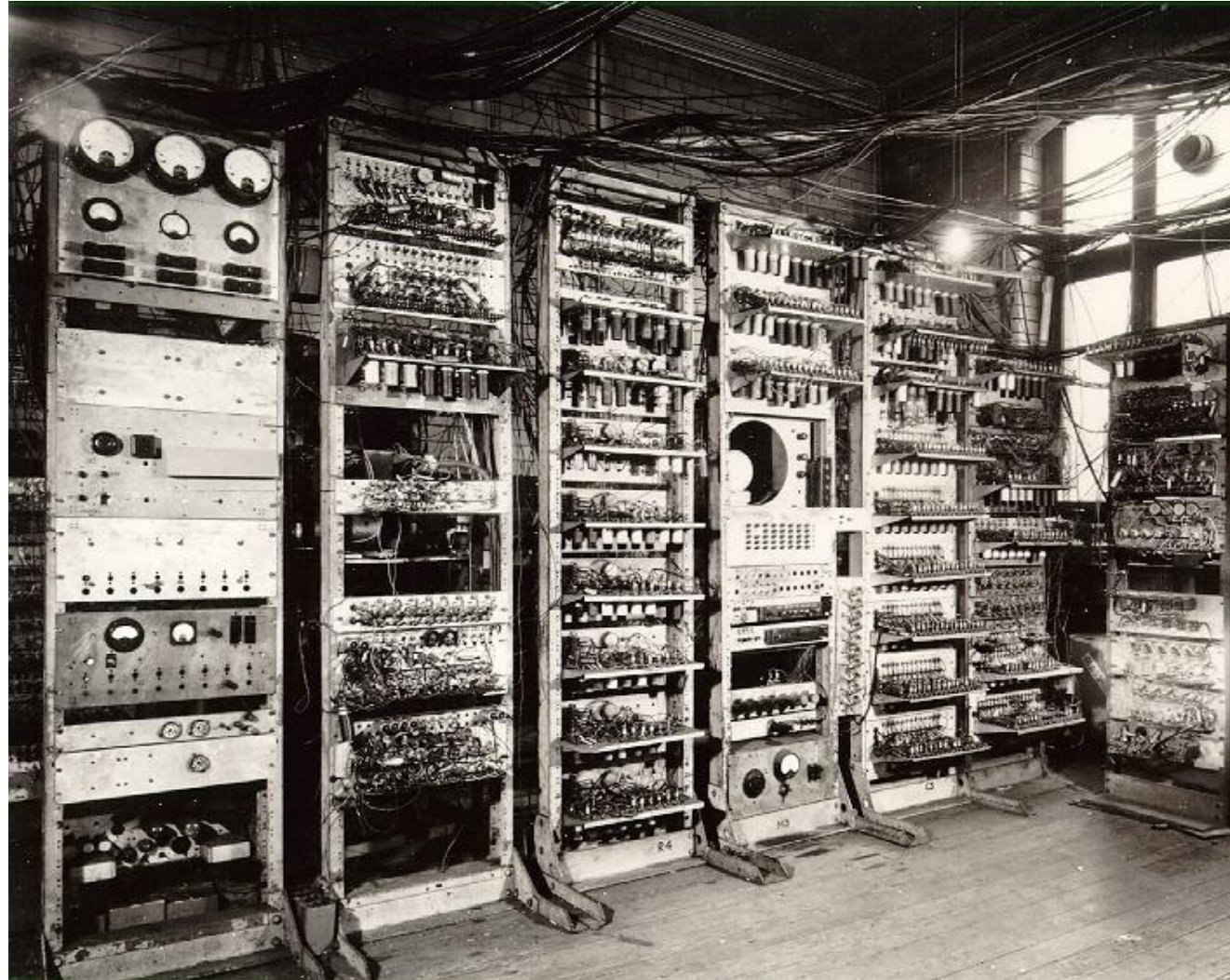
First actual case of bug being found.

1630 Antan started.

1700 closed down.

Relay 2145
Relay 3370

Manchester Mark 1



1950 – 1960

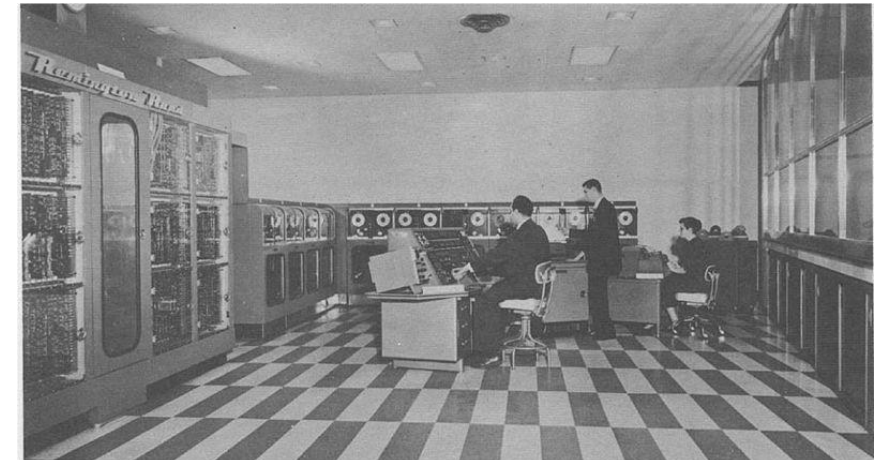
First industrial application

After the initial research project of the last decade, computers now made it into industry

- Univac I was designed for administrative use
- Installations at US Air Force, Prudential Insurance Company, US Steel,...
- The IBM 701 was the first commercial scientific computer (2k of memory)

In parallel, first “real” programming languages were developed

- First generation of machine code (i.e. binary numbers) where replaced by assembly instructions
- Macro programming was introduced
- First high level languages like Fortran, LISP and COBOL



Univac I at Franklin Life Insurance Company

The software crisis

- With more powerful machines available a “software crisis” became apparent
- The term was coined at the first NATO Engineering conference in 1968
- *”The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.”* (Edsger Dijkstra, The Humble Programmer (EWD340), Communications of the ACM, 1972)



1960 – 1975

The new age of programming

- Simula, invented in the late 1960s by Nygaard and Dahl as a superset of Algol 60, was the first language designed to support object-oriented programming.
- Smalltalk (mid 1970s) provided a complete ground-up design of an object-oriented language.
- C, an early systems programming language, was developed by Dennis Ritchie and Ken Thompson at Bell Labs between 1969 and 1973.
- Prolog, designed in 1972 by Colmerauer, Roussel, and Kowalski, was the first logic programming language.
- ML built a polymorphic type system (invented by Robin Milner in 1973) on top of Lisp, pioneering statically typed functional programming languages.

Listing - MC68HC11E9 Bootloader (With Security) Sheet 2 of 5

```
bf40                ORG    $BF40

bf40                BEGIN  EQU    *
                    * INIT STACK
                    LDS     #$01FF
                    * INIT X REG FOR INDEXED ACCESS TO REGISTERS
bf43 ce 10 00        LDX     #$1000
                    * PUT PORT D IN WIRE OR MODE
bf46 1c 28 20        BSET   SPCR,X$20
                    * INIT SCI AND RESTART BAUD DIVIDER CHAIN
bf49 cc a2 0c        LDD     #$A20C      DIV BY 16
bf4c a7 2b          STAA   BAUD,X
                    * RECEIVER & TRANSMITTER ENABLED
bf4e e7 2d          STAB   SCCR2,X

                    * TEST THE SECURITY BIT
bf50 1e 3f 08 35     BRSET  CCR1G,X $08  NOSEC

                    *****
                    * WE ARE IN SECURITY MODE
                    * OUTPUT 1 ON TRANSMITTER
                    AGAIN EQU    *
bf54 a6 2e          LDAA   SCSR,X
bf56 86 ff          LDAA   #$FF
bf58 a7 2f          STAA   SCDAT,X
                    * ACCA NOW IS SET FOR $FF

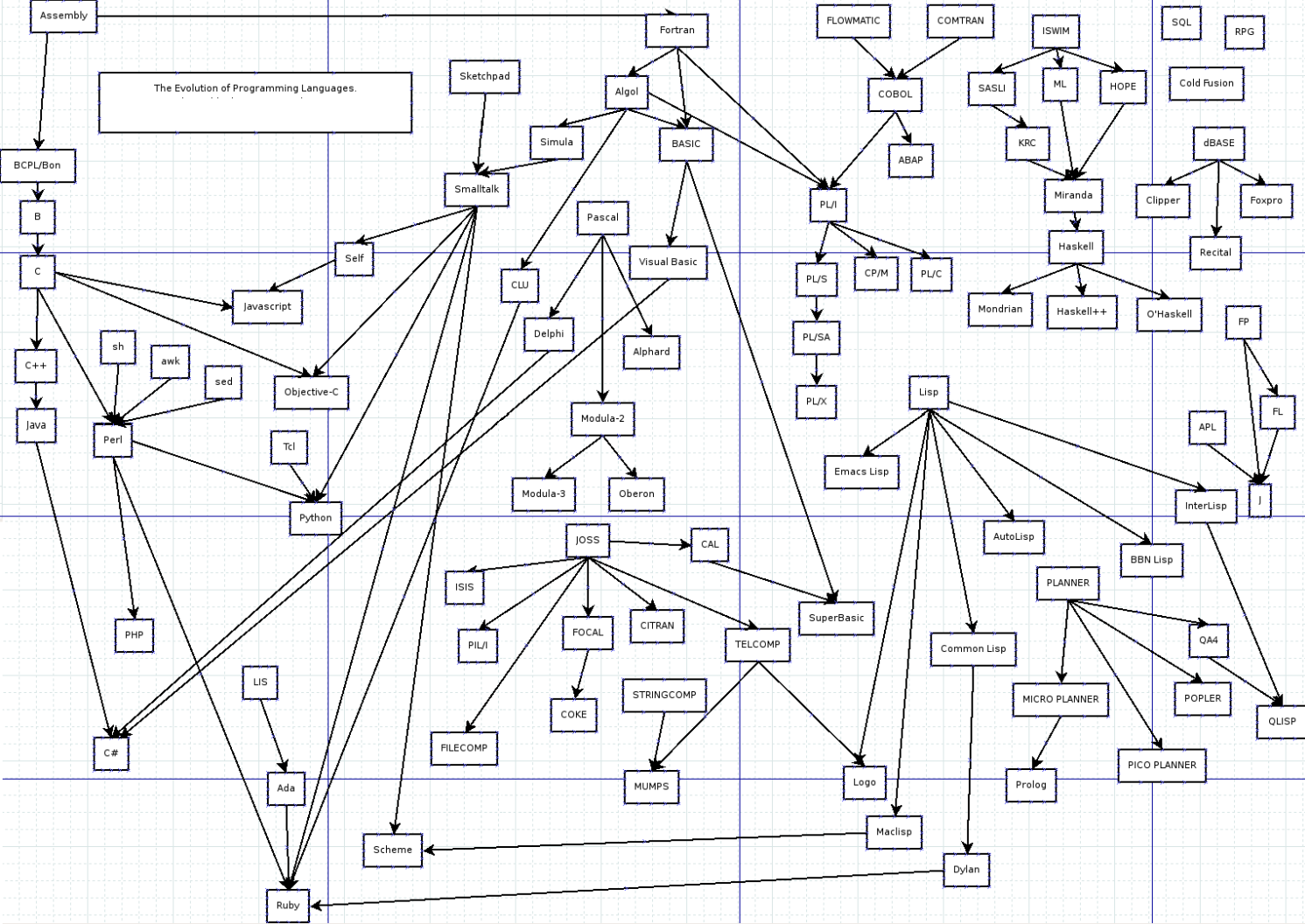
                    * ERASE EEPROM:
                    * TURN OFF BLOCK PROTECT
bf5a 6f 35          CLR    BPROT,X
                    * SET ERASE AND ELAT BITS BEFORE USING "ERASE"
                    *          CHANGE $0C TO $06
bf5c 54            LSRB
                    STAB   PPRG,X
bf5d e7 3b          STAB   EEPSTR
bf5f f7 b6 0        BSR    ERASE
bf62 8d 72          WRITE EEPROM LOCATION

                    * ACCB IS NOW SET FOR $06
                    * ERASE CYCLE IS COMPLETE
```

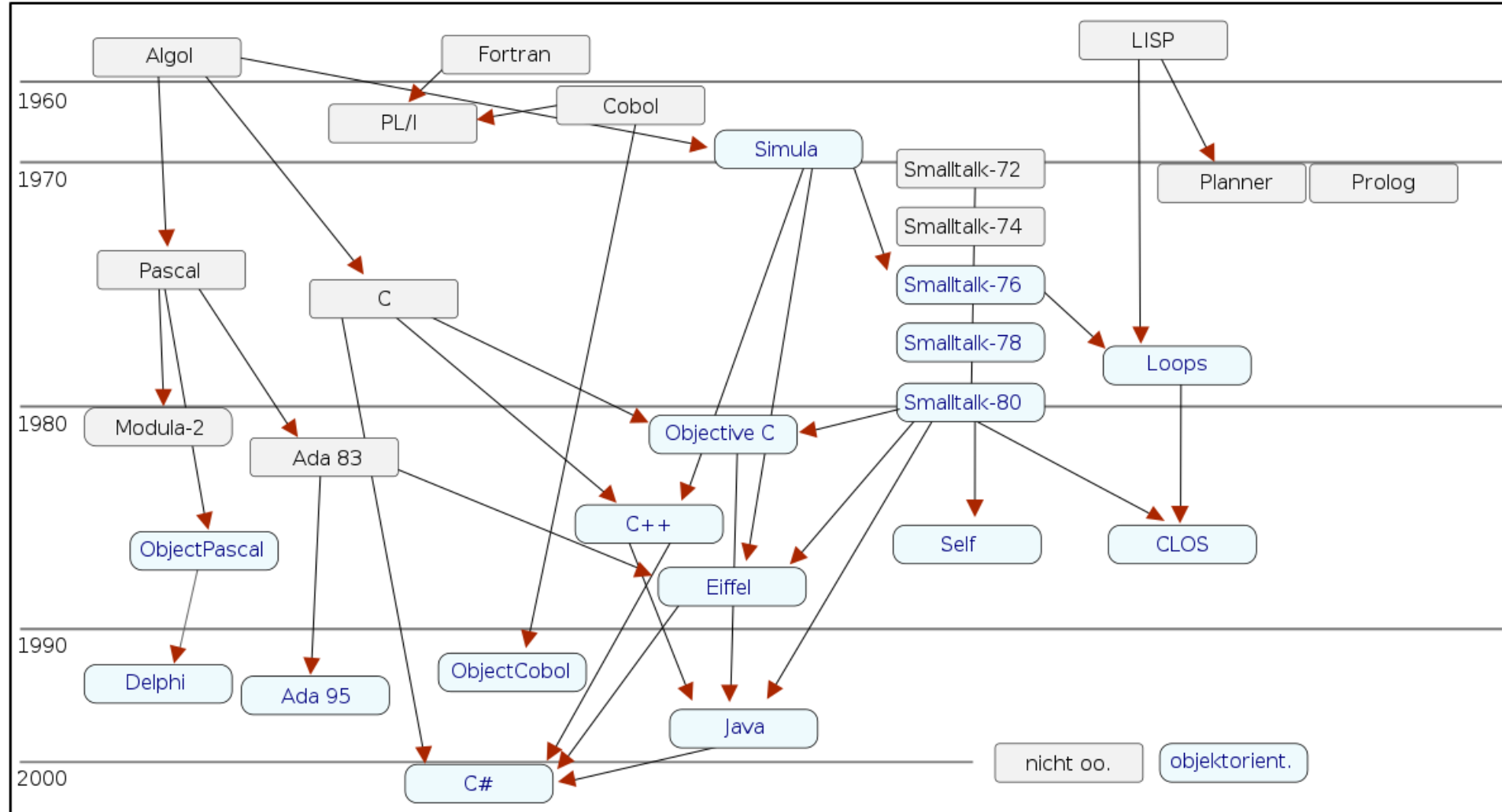
Example of a Fortran program

```
C AREA OF A TRIANGLE WITH A STANDARD SQUARE ROOT FUNCTION
      READ INPUT TAPE 5, 501, IA, IB, IC
      501 FORMAT (3I5)
C IA, IB, AND IC MAY NOT BE NEGATIVE
      IF (IA) 777, 777, 701
701 IF (IB) 777, 777, 702
705 IF (IB+IC-IA) 777,777,799
777 STOP 1
C USING HERON'S FORMULA WE CALCULATE THE AREA OF THE TRIANGLE
799 S = FLOATF (IA + IB + IC) / 2.0
      AREA = SQRT( S * (S - FLOATF(IA)) * (S - FLOATF(IB)) *
+ (S - FLOATF(IC)))
      WRITE OUTPUT TAPE 6, 601, IA, IB, IC, AREA 601
      FORMAT (4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2, + 13H SQUARE UNITS)
      STOP END
```

History of programming languages



Object Oriented programming languages



Most successful from this area: C

C is a

- general-purpose
- cross-platform
- block structured
- procedural

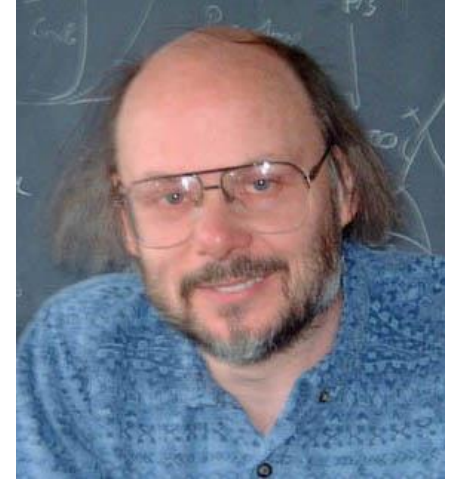
programming language

- C was developed in 1972 by Dennis Ritchie and Ken Thompson at the Bell Telephone Laboratories for use with the Unix operating system.
- 1978 “The C programming language”
- 1988, 1998 and 2011 Standardization as ANSI-C



Object oriented C: C++

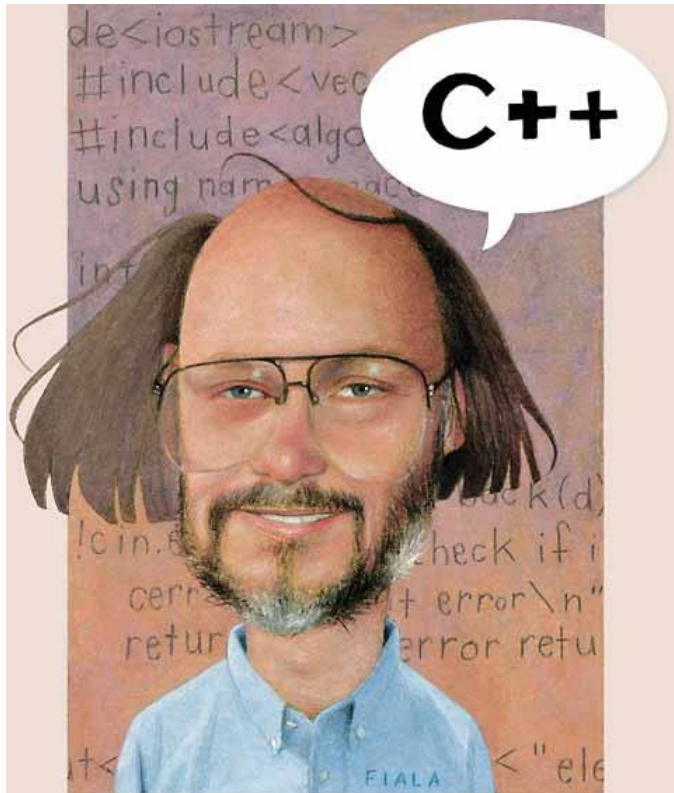
- Developed by Bjarne Stroustrup between 1979 and 1983 at the Bell Labs
- Originally named “C with classes”
- Additional object oriented features
 - Classes
 - Virtual functions, operator overloading
 - Multiple inheritance, templates
 - Exception handling
 - Focus on run-time performance
- Standardized as ISO/IEC 14882:1998, 2003 and 2011
- Version used in lecture: C++11



Is C++ the solution?

C makes it easy to shoot yourself in the foot; C++ makes it harder; but when you do it blows your whole leg off.

Bjarne Stroustrup



Don't ignore compiler warnings!

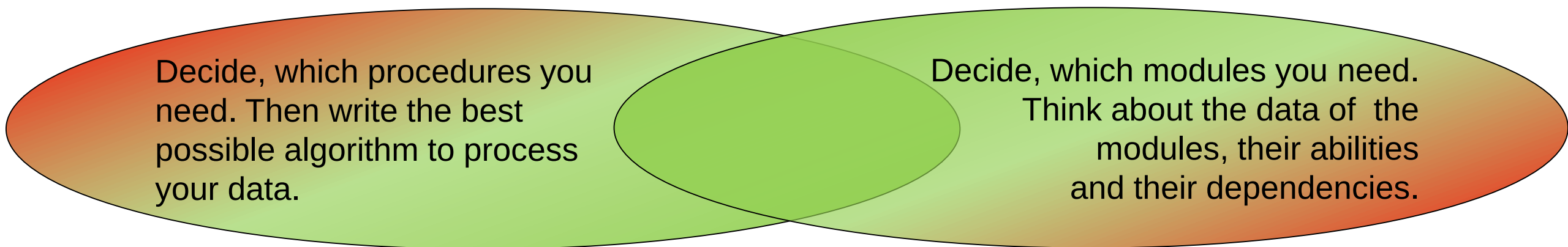
Difference between functional and object oriented perspective

Functional programming

- Very technical perspective
- Data and functions are separated
- Limited modelling support
- Limited encapsulation, generalisation, specialisation capabilities
- Limited language features

Object oriented programming

- Problem focussed, abstract view
- Data and functions are encapsulated in a class
- Good modelling support (UML)
- Inheritance, templates and other generalisation concepts
- Strong language features (incl. libraries)



Decide, which procedures you need. Then write the best possible algorithm to process your data.

Decide, which modules you need. Think about the data of the modules, their abilities and their dependencies.

Example: Space Invader

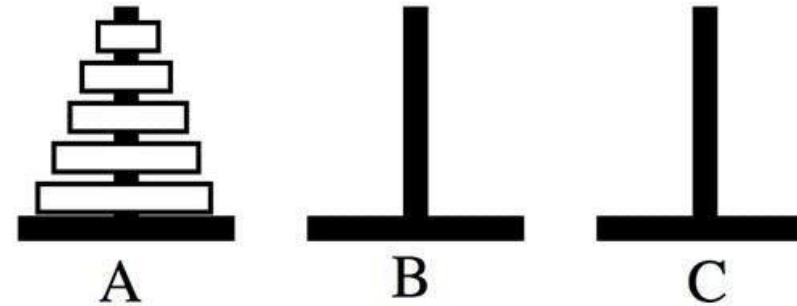


1978 – My first computer



And my first programs...

- Hello World - 10 print “Hallo Peter”
- Pocket Calculator
- Tower of Hanoi
 - Failure
 - No clue about recursion
- Kingdom Simulator “Hammurabi”
 - Economical Simulation
 - It worked – somehow
 - Model-View-Control – what the hell



```
H  H      D RRRRR
      D  RRRR
      H      H  D  RRRRR
      D  RRRRRR
FFFFF      D  RRRRRR
FFFFFF      D  RRRRRR
```

The Millenium Language isn't everything



Situation around the year 2000

- Powerful programming languages available
- Integrated IDE's available
- Training / teaching available

Standish Report 1994:
85% of all software projects fail !!!

Increasing interest in Software Engineering
Processes and CASE tools

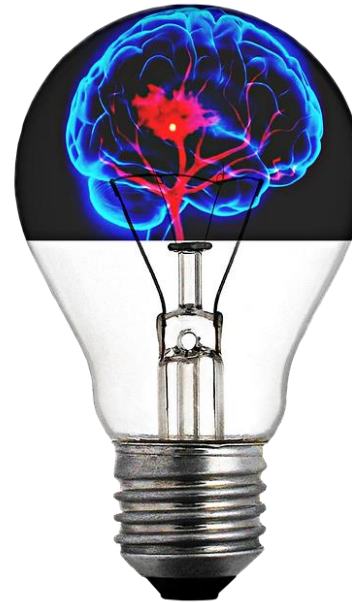
Key issues

Many software projects failed

- Not because of programming language limitation
- but due to wrong project organization (underestimated effort, complexity)
- and limited communication in bigger teams
- and lack of proper software engineering
- in combination with code rot!

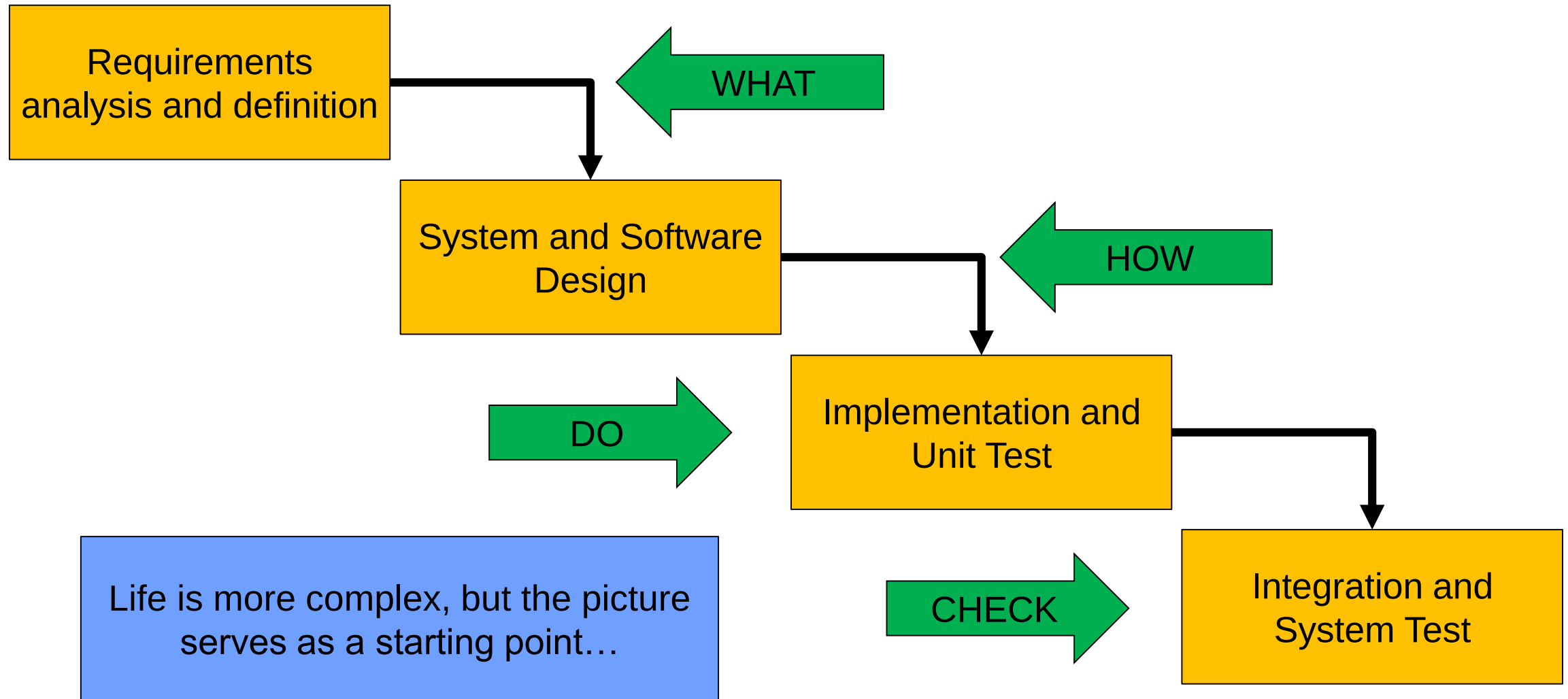
Awareness

- Project success depends on heroes
- “From the brain into the keyboard” approach does not work



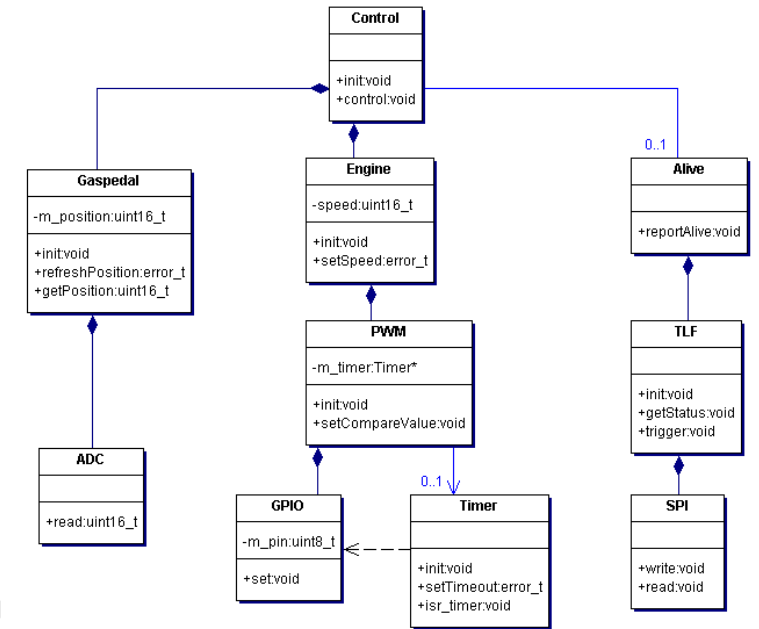
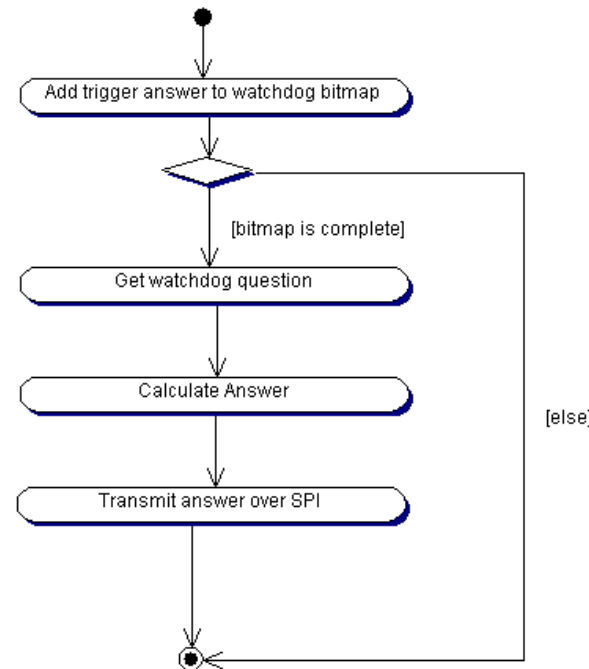
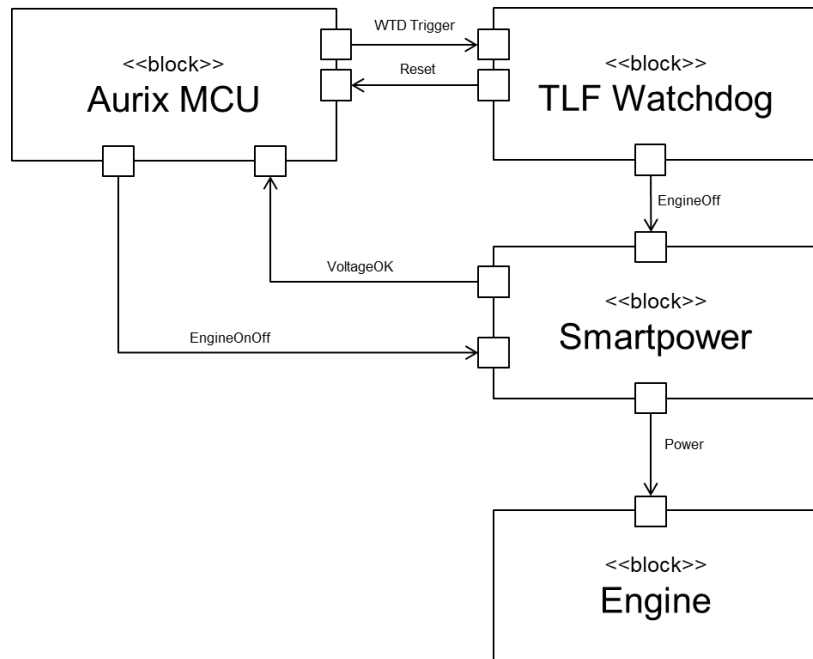
First process frameworks

Waterfall: 1970, Royce



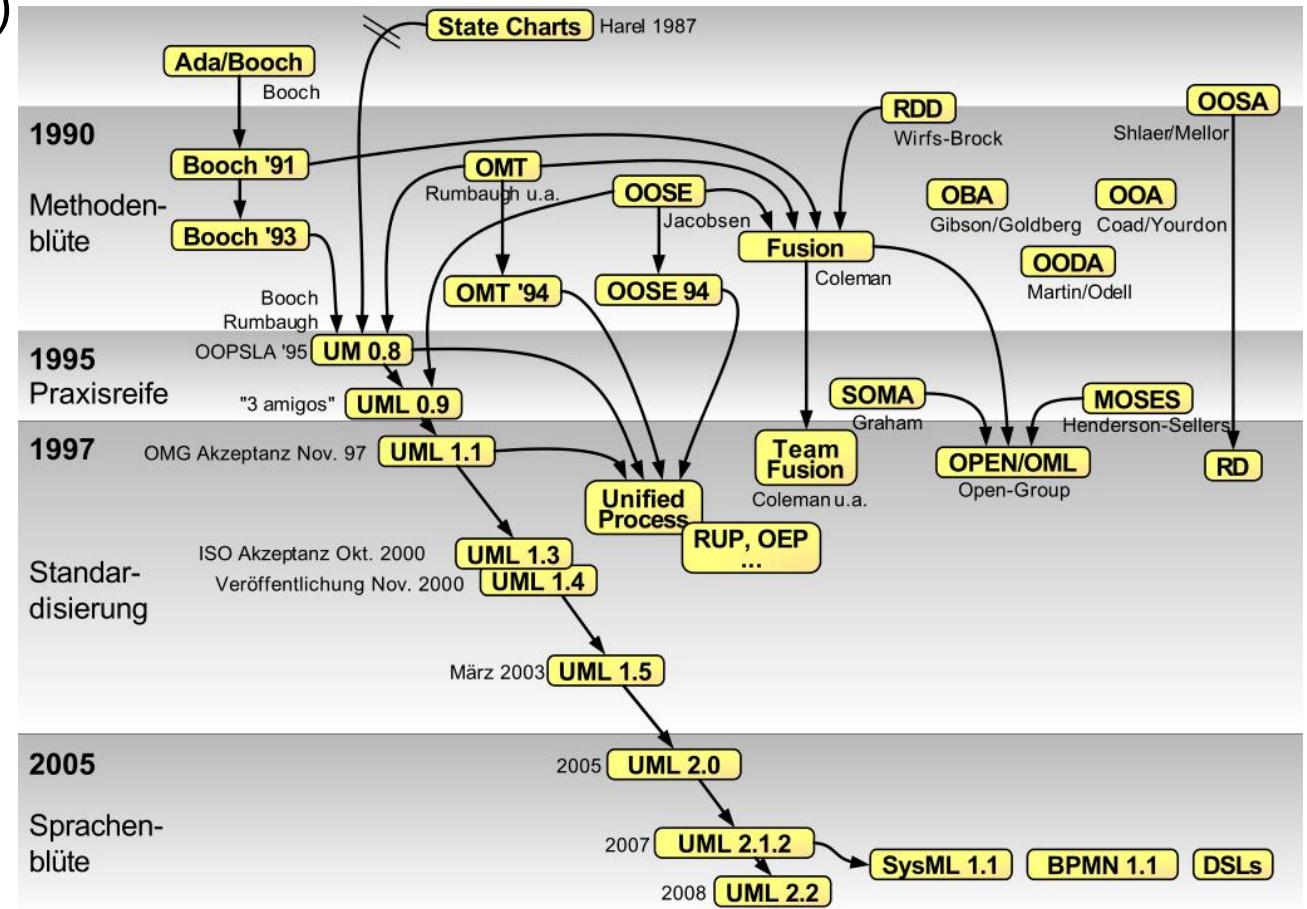
Modelling

- Humans tend to get an idea from a picture faster than from a lengthy text
- ➔ Use graphics notations for models
- ➔ Provide different views (level of abstraction, static or dynamic focus,...)

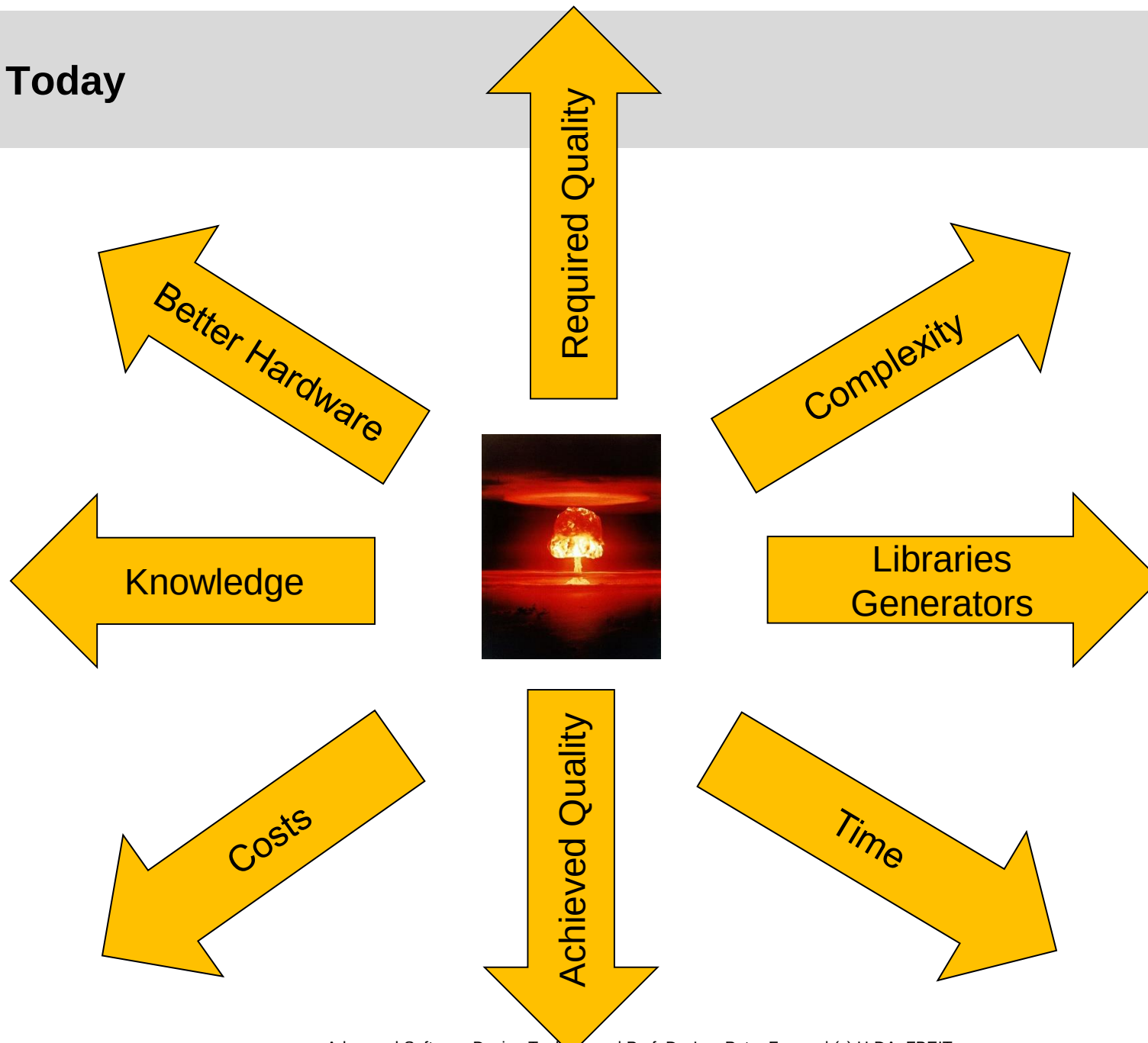


Modelling History

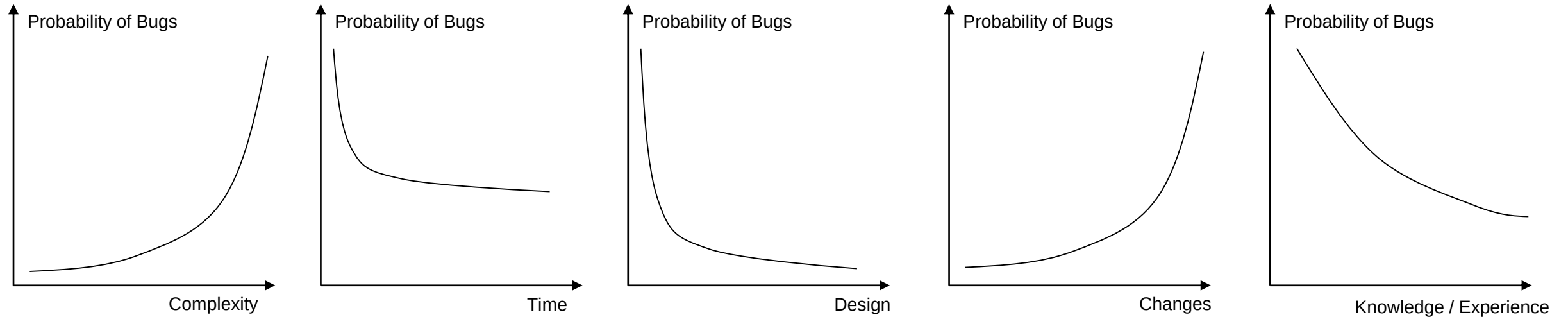
- Standardize notation for really meaningful graphics
- ➔ UML (Unified Modelling language)
- ➔ SysML (System Modelling Language)



Challenges Today



Some fundamental correlations – and dependencies



WE OFFER 3 KINDS OF SERVICES
GOOD-CHEAP-FAST
BUT YOU CAN PICK ONLY TWO

GOOD & CHEAP WON'T BE **FAST**
FAST & GOOD WON'T BE **CHEAP**
CHEAP & FAST WON'T BE **GOOD**

Wrap-Up

- A brief history of computer and programming language development
 - Ada Lovelace's first program
 - Turing machine
 - Programming Languages
 - Software Engineering
 - Software development process
 - Modelling

Computers and programming haven't changed that much over the last decade....

Very likely, there the rules for building a clean architecture also remain universal and changeless...